

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

C05: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Annexure A

**Duration : 1 Hour
Total Marks:50**

Design Task

Code of Conduct:

I RAJESWARI P-192311072 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RAJESWARI P

Design Task

1. Hybrid AI Recommendation Engine Design

Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. Smart Disaster Detection System

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. AI-Based Fraud Detection Framework

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. Autonomous Supply Chain Optimization System

Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. Personalized Learning Analytics Platform

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

1. Hybrid AI Recommendation Engine Design

A Hybrid AI Recommendation Engine combines **Content-Based Filtering** and **Collaborative Filtering** to provide personalized recommendations. User-item interaction data such as ratings, searches, clicks, and

purchases are stored in distributed datasets. Using **PySpark RDDs**, large datasets can be divided into partitions and processed in parallel across multiple nodes. RDD transformations such as `map()`, `filter()`, `flatMap()`, `groupByKey()`, and `reduceByKey()` are used to calculate user preferences, item similarities, and recommendations. Content-based filtering recommends items similar to those previously preferred by the user, while collaborative filtering recommends items based on the behavior of similar users. Intelligent agents continuously monitor user actions and feedback and update the recommendation model. For example, an agent can detect a change in user interest and adjust recommendations in real time. Thus, the system provides scalable, adaptive, and personalized recommendations.

2. Smart Disaster Detection System

A Smart Disaster Detection System uses **PySpark and AI** to identify natural disasters such as floods and earthquakes from multiple data sources. Satellite images, IoT sensor readings, weather data, and social media posts are collected and converted into distributed RDDs. RDD operations such as `map()`, `filter()`, `join()`, and `reduceByKey()` process the data in parallel. **Data fusion** combines information from different sources to improve detection accuracy. For example, sensor readings can be combined with satellite images and social media reports to confirm a flood. AI models detect abnormal patterns and classify the type and severity of the disaster. Intelligent agents coordinate different tasks such as data collection, disaster detection, location identification, and emergency notification. When a critical event is detected, agents can trigger early warnings and recommend evacuation or emergency response strategies. The distributed architecture enables fast processing of large-scale disaster data.

3. AI-Based Fraud Detection Framework

An AI-Based Fraud Detection Framework uses **PySpark RDDs** to process millions of financial transactions efficiently. Transaction data such as amount, location, time, account activity, and transaction frequency are stored as distributed RDDs. Operations such as `map()`, `filter()`, `groupByKey()`, and `reduceByKey()` are used to extract transaction features and identify unusual behavior. Anomaly detection algorithms compare current transactions with normal transaction patterns. Transactions with unusual amounts, locations, frequencies, or behavior are assigned higher fraud scores. Intelligent agents monitor transactions continuously and collaborate to identify suspicious patterns. One agent may monitor transaction behavior, another may analyze customer history, and another may manage alerts. Suspicious transactions generate alerts for further investigation. Feedback from confirmed fraud and legitimate transactions is used to improve the detection model continuously. This provides a scalable and adaptive fraud detection system.

4. Autonomous Supply Chain Optimization System

An Autonomous Supply Chain Optimization System uses **PySpark and intelligent agents** to optimize inventory, transportation, and demand forecasting. Large-scale data such as sales records, inventory levels, supplier information, transportation data, and customer demand are stored as RDDs. PySpark transformations such as `map()`, `filter()`, `join()`, and `reduceByKey()` process the data across multiple nodes. AI models analyze historical sales data to forecast future demand and determine appropriate inventory levels. Multiple intelligent agents work independently but communicate with each other. For example, an **Inventory Agent** monitors stock levels, a **Logistics Agent** selects efficient transportation routes, and a **Demand Agent** forecasts customer demand. Agents exchange information and make decentralized decisions to reduce transportation costs, avoid stockouts, and minimize excess inventory. Continuous feedback allows the system to adapt to changes in demand and supply conditions.

5. Personalized Learning Analytics Platform

A Personalized Learning Analytics Platform uses **AI, PySpark, and RDDs** to analyze student learning behavior and performance. Data such as quiz scores, assignment marks, course progress, login activity, and learning time are collected and stored in distributed RDDs. PySpark operations such as `map()`, `filter()`, `groupByKey()`, and `reduceByKey()` process data from thousands of students in parallel. AI models identify each student's strengths, weaknesses, learning patterns, and performance levels. Intelligent agents continuously monitor student activities and provide personalized recommendations. For example, an agent can recommend additional exercises when a student performs poorly in a particular topic. Another agent can provide real-time feedback and adjust the difficulty of learning materials. The platform continuously learns from student feedback and performance. Therefore, the system provides **scalable, adaptive, and personalized learning** for large numbers of students.

RUBRICS FOR CALCULATION

Evaluation Criteria	Problem Understanding & Requirement Analysis	System Architecture Design	Use of PySpark & RDD Operations	Intelligent Agent Integration	Scalability & Distributed Computing Design	Data Flow & Processing Pipeline	Innovation & Creativity	Clarity, Presentation & Documentation
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	Excellent (5)	Good (4)	Average (3)	Poor (2)
Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect

System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation